PROJECT SHOWCASE / ENGINEERING & PRODUCT PORTFOLIO

Complete Project Portfolio

Technical Case Studies

Six projects: MetricForge · Petinha · Gig Goal Tracker · Bills Reminder · MathQuest Arcade · Career Town (CV RPG)

Prepared for professional review | November 4, 2026 | English edition

Evidence note: Based on prior project discussions, described architecture, selected development/debugging history and portfolio records available at preparation time. This is a portfolio case study, not a fresh source-code audit, external security review, market study or verified commercial due diligence. Planned capabilities and unverified outcomes are labeled accordingly.

Contents

- 1. MetricForge - Social analytics SaaS
- 2. Petinha - Virtual pet / interactive web game
- 3. Gig Goal Tracker - Gig-work earnings productivity
- 4. Bills Reminder - Recurring payment organization
- 5. MathQuest Arcade - Educational logic / math games
- 6. Career Town (CV RPG) - Interactive resume / portfolio

How to use this dossier

For hiring discussions, use the technical edition to examine component responsibilities, design decisions, implementation evidence and areas worth demonstrating in code. For product or sponsorship meetings, use the business edition to evaluate problem framing, proposed value, distribution hypotheses, validation and principal risks. The corresponding per-project PDF can be attached to each project page.

MetricForge | Technical Case Study

Social analytics SaaS | Stage: Working application with YouTube integration; broader platform scope evolving

Technical summary

MetricForge is a multi-user social analytics platform intended to centralize account data, media performance, and analytical insights across social channels. Its confirmed development history includes authentication, a protected analytics dashboard, GraphQL domain models, MongoDB persistence, and YouTube account authorization and synchronization.

Problem and engineering context

Creators and small organizations must move between platform dashboards to correlate audience, content, engagement, and performance. Context becomes fragmented, comparison is expensive, and historical snapshots are difficult to standardize.

Technology stack

- Next.js 16 + React and TypeScript for application UI and server endpoints.
- GraphQL Yoga and Apollo Client for typed query/mutation workflows.
- MongoDB Atlas / Mongoose for users, connected accounts, content records, metric snapshots and analytics-related entities.
- YouTube OAuth and YouTube API synchronization; credentials-based access with password hashing and cookie/session mechanisms discussed.
- Docker deployment and Caddy-based reverse proxy on a VPS; production configurations require secret and callback management.

System architecture and component boundaries

- Web interface: authenticated dashboards and analytical views backed by GraphQL queries.
- Application boundary: authentication and integration routes, GraphQL resolvers, synchronization jobs and domain services.
- Persistence: collections for users, social connections/accounts, content items, historical metrics, comments and insight-related entities.
- Integration pipeline: obtain permitted channel access, fetch source records, normalize payloads, persist snapshots, then aggregate KPIs.
- Deployment: containerized application with reverse proxy and remote MongoDB Atlas; network allowlists, OAuth redirects and environment secrets are operational dependencies.

Data model and domain contracts

- User and authorization context: isolate connected social accounts per user and enforce ownership in each resolver.
- SocialConnection/SocialAccount: external provider identifiers, token metadata, owner and synchronization state.
- ContentItem and metric snapshots: timestamped platform metrics, content metadata and aggregation inputs.

- Comments/insights: text-derived analytical fields where actually available; distinguish computed analysis from native platform information.
- Analytics: engagement rate and an application-level metricForgeScore discussed in development; formula, sampling and validation must be documented before external performance claims.

Primary application flows

- User registration and login; access protected pages and a personalized dashboard.
- Connect a YouTube channel via OAuth; persist authorization safely and handle expired/denied permissions.
- Run synchronization for channel/content/comments/metrics, then display selected dashboard KPIs.
- Read and filter data through the GraphQL domain; protect every operation with authorization and appropriate input validation.

Engineering challenges and tradeoffs

- OAuth callback correctness across local and production environments; redirect URI consistency.
- MongoDB Atlas network access and secret management during VPS deployment.
- Potential duplicated or stale metric snapshots when sync jobs retry; idempotency and upsert keys are important.
- Provider quotas, pagination, rate limits and partial synchronization require observability and backoff.
- Cross-provider normalization can hide incomparable definitions; metrics must retain provenance and provider-specific meaning.

Available evidence and limits

- Previously discussed production domain and Docker deployment, build routes and YouTube-specific sync flows.
- Prior code/context describes GraphQL entities, account ownership, KPIs and metric snapshots.
- YouTube is the directly supported integration in available context; Facebook and Instagram were modeled but are not verified as fully synchronized.
- No audited performance benchmarks, paid customer figures, or independent security assessment supplied.

Recommended technical validation / next steps

- Add integration-level tests for OAuth expiration, permission revocation, pagination and sync failures.
- Instrument job metrics, lag, rate-limit counters and data freshness labels.
- Formalize authorization tests for tenant isolation and secrets-at-rest policy.
- Validate cross-provider metric definitions before claiming comparable dashboards.
- Only present FREE/PRO/BUSINESS as commercial tiers if payment, entitlements and operations are actually implemented.

Recruiter interview prompts

- Explain an end-to-end YouTube synchronization and where idempotency is enforced.
- How do you authorize GraphQL resolvers to prevent user-account data leakage?
- How would you handle provider quotas and stale metrics?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.

Petinha | Technical Case Study

Virtual pet / interactive web game | Stage: Existing application under active engine migration and movement/animation debugging

Technical summary

Petinha is an interactive virtual-pet experience with state-based needs and player actions. Development discussions cover a Next.js application backed by MongoDB and authentication, while the game runtime is being migrated toward Phaser 3 so movement, animation and object interactions are owned by a coherent game loop.

Problem and engineering context

Interactive virtual pets must consistently coordinate persistent needs, character animations, inventory-like actions and scene interactions. Mixing DOM/CSS animations with game-engine transforms can introduce divergent state and unpredictable movement.

Technology stack

- Phaser 3 as the targeted runtime for scenes, sprites, animation, camera and movement.
- TypeScript and Next.js/React, with React intended primarily for authentication after migration.
- NextAuth / Google sign-in and MongoDB-backed pet/account state.
- Phaser scene modules including PetHomeScene and interaction helpers such as WorldInteraction.
- Spritesheets and assets organized by pet or avatar state, including separate gender-specific artwork.

System architecture and component boundaries

- Authentication shell: identify the account and supply an authorized session to the game layer.
- Game scene: Phaser owns input, entity positions, collision, animation playback and lifecycle events.
- Interaction subsystem: resolve hitboxes, proximity and object state transitions (e.g., fridge and microwave).
- State synchronization: translate authoritative pet state to scene visuals and persist actions without losing transitions.
- Asset pipeline: preload and validate spritesheet naming, frame ranges, origin, dimensions and animation maps.

Data model and domain contracts

- Pet state: hunger, happiness, energy, health, cleanliness/sleep-related state and progression.
- SceneView and spriteToken are an interface between application state and visual representation.
- Interaction states: carried item, selected target, animation state, activity completion and guarded transition.
- Potential event history: timestamps/actions for debugging or recovering from interrupted interactions; confirm whether persisted.

Primary application flows

- Sign in and load the user-specific pet.
- Explore the room and interact with world objects.
- Perform actions such as feeding, playing, cleaning, medicine or sleep; update states.
- Select the correct animation for gender, pet condition and action, then render consistently in Phaser.

Engineering challenges and tradeoffs

- Movement oscillation and apparent freezes after fridge/microwave interactions were reported.
- Character art briefly swaps between male/female or dirty/normal sprites; spriteToken mapping and cache keys need tests.
- Missing sprite paths causing HTTP 404; asset manifest and preload validation are essential.
- Dynamic hitbox typing and scene-state nullability triggered TypeScript build issues.
- React/DOM and Phaser ownership overlap can create double-rendered or contradictory motion.

Available evidence and limits

- Reported working pet interactions and existing NextAuth/MongoDB application architecture.
- Phaser migration files, animation debugging and movement bugs were explicitly discussed.
- A fully completed Phaser-only architecture, a stable build and public player adoption have not been verified.

Recommended technical validation / next steps

- Complete a single authoritative movement and interaction controller inside Phaser.
- Build deterministic reproduction tests for fridge-to-microwave-to-carried-item transitions.
- Add asset manifest validation and sprite-change debug telemetry.
- Define a finite-state-machine contract for pet actions, interruption and persistence.
- Profile mobile performance, memory use and animation lifecycle before publishing claims.

Recruiter interview prompts

- How is state ownership divided among Phaser, React and persistence?
- Explain how a finite-state machine would prevent the reported movement oscillation.
- How do you validate sprite assets and correct animation selection?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.

Gig Goal Tracker | Technical Case Study

Gig-work earnings productivity | Stage: Actively developed earnings/goal tracking app; advanced mileage/tax reporting scoped or evolving

Technical summary

Gig Goal Tracker is a mobile-friendly web application to help gig workers manage earning targets, trips and work sessions. It is grounded in the developer's direct use of gig delivery workflows and aims to keep essential daily tracking simple while progressively revealing advanced analysis.

Problem and engineering context

Gig workers often evaluate offers, maintain earnings goals and assemble tax records across fragmented tools. Without a consistent session and trip history, it is difficult to interpret work performance or reconcile mileage.

Technology stack

- Next.js 16, React 19 and TypeScript for application and UI.
- MongoDB / Mongoose for user settings, trip entries, work sessions, earning records and goals.
- Email/password authentication; Google sign-in and account recovery discussed as product evolution.
- Docker deployment on a VPS, with application-specific environment variables and secrets.
- Progressive-disclosure UX to keep the baseline workflow lightweight.

System architecture and component boundaries

- Dashboard: current goal, progress, earning summaries and quick entry actions.
- Settings: workdays, configurable sessions and target windows.
- Transaction records: trip earning, tips, timestamp and optional miles.
- Aggregation: day/week/month/year totals and efficiency ratios using consistent inclusion rules.
- Optional modules: odometer-based mileage and tax exports, isolated from core daily tracking.

Data model and domain contracts

- User preferences: weekly goal, work schedule, break handling and session definitions.
- Trip and earnings record: gross amounts, tips, platform, timestamps and editable metadata.
- Mileage: hybrid design using daily odometer as a primary log and per-trip miles as analytical detail (planned/evolving).
- Derived calculations: dollars/hour, dollars/mile, goal attainment and period totals; ensure time and mileage denominators are correct.
- Reporting: exportable records should distinguish business miles, other miles, tips and source completeness.

Primary application flows

- Choose goal and configure working days/sessions.

- Start work and record trips and tips; edit past entries with an audit-friendly policy.
- Track progress at a glance and review historical summaries.
- Optionally enable advanced mileage/tax tracking when the user needs it.

Engineering challenges and tradeoffs

- Time-zone boundaries and workdays crossing midnight can shift weekly totals.
- Double-counted cash tips or duplicated trips can distort analytics.
- Manual odometer entries require validation for decreases, gaps and inconsistencies.
- Financial summaries must be auditable and should not imply official tax filing or advice.
- Responsive density must preserve simple task entry for workers on the move.

Available evidence and limits

- Project discussions confirm an actively used/developed goal and earnings workflow and current files under iteration.
- A proposed optional odometer/taxes module and future statistics were discussed; do not treat them as finished.
- No independently verified customer count, subscription processing or certified tax integration.

Recommended technical validation / next steps

- Define canonical trip/session schema and date/time boundary rules.
- Implement mileage reconciliation and explicit missing-data indicators.
- Test money arithmetic with integer cents and reproducible aggregation fixtures.
- Create CSV/PDF exports with a clear non-tax-advice disclaimer.
- Private invitation beta, then evaluate whether Stripe subscriptions are justified.

Recruiter interview prompts

- How are weekly boundaries and session crossing-midnight handled?
- How do you prevent duplicated trips and ensure reliable money arithmetic?
- What schema supports odometer reconciliation and export?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.

Bills Reminder | Technical Case Study

Recurring payment organization | Stage: Backend/data model and reminder scheduling worked on; complete feature parity unverified

Technical summary

Bills Reminder is a personal payment-organization application intended to consolidate recurring bill schedules, payment status and proactive notifications. Development history includes MongoDB Atlas collections, payment records, notification logs and work on job locking for scheduled processing.

Problem and engineering context

Recurring bills vary in frequency, due date and status. Manual calendars and disconnected inbox reminders can lead to missed payments, poor visibility and duplicate alerts.

Technology stack

- Next.js-based web administration and MongoDB Atlas persistence discussed in project history.
- Bill, BillPayment, NotificationLog and BillOverride domain models.
- Scheduler/job-lock approach for processing upcoming bill notifications.
- Email delivery workflow and weekly digest concept.
- Docker/VPS deployment context alongside other user applications.

System architecture and component boundaries

- User-facing bill management for recurring schedules, status and history.
- Billing domain: recurring bill definition separated from individual payments/overrides.
- Scheduler: select due reminders, acquire a short-lived job lock, emit notifications and log outcomes.
- Persistence: maintain a per-user boundary and normalized date/time semantics.
- Notification subsystem: idempotency controls and retryable delivery recording.

Data model and domain contracts

- Bill: owner, merchant label, amount, cadence, due date and notification preferences.
- BillPayment: amount/date/payment status for a specific recurrence.
- BillOverride: exceptional due date or amount changes for one instance.
- NotificationLog: delivery intent, channel, outcome, retry count and deduplication key.
- Job lock: owner, lock expiration and TTL index to support safe concurrent schedules.

Primary application flows

- Create a recurring obligation and choose notification preferences.
- Compute the next due occurrence including exceptions.
- Scheduler evaluates alert windows, acquires lock and records notification outcome.
- Mark a bill as paid; show history without destroying the recurring bill definition.

Engineering challenges and tradeoffs

- A previously observed joblocks index lacked the intended TTL expireAfterSeconds setting; final correction was not confirmed.
- Multiple scheduler instances may issue duplicate emails without atomic lock and deduplication.
- Monthly recurrence edge cases: 29th-31st, leap years, DST and calendar changes.
- Email provider failures require retry/backoff and a defensible sending log.
- Sensitive financial reminders require careful permissions, retention and encryption policies.

Available evidence and limits

- Prior MongoDB shell checks and named collections for bills/payments/overrides/notifications.
- TTL locking investigation directly discussed; final deployed behavior was not verified.
- CSV import, complete dashboard and polished production notification system cannot be claimed without source review.

Recommended technical validation / next steps

- Test index shape with actual MongoDB metadata and TTL expiry.
- Unit test recurrence generation and exception behavior over multiple years.
- Integration test concurrent schedulers for duplicate delivery.
- Add preview/dry-run mode, observability and privacy controls.
- Validate value beyond built-in banking and calendar alerts.

Recruiter interview prompts

- What prevents duplicate notifications under concurrent scheduled workers?
- How are month-end recurrences and overrides calculated?
- What is the correct MongoDB TTL configuration and its limits?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.

MathQuest Arcade | Technical Case Study

Educational logic / math games | Stage: Playable mini-game work and extensive visual design; full catalog and polish evolving

Technical summary

MathQuest Arcade is a web portal for mathematical reasoning and logic mini-games, built around an illustrated medieval tavern hub. Users select an avatar and access experiences such as Cows and Bulls and Hidden Tiles. The project emphasizes approachable learning-by-doing and a coherent visual world.

Problem and engineering context

Static math practice can feel abstract and repetitive. A game-based collection may encourage exploration, iteration and reasoning through bite-sized challenges without requiring a long learning session.

Technology stack

- React 19, TypeScript and Vite for the portal interface.
- Phaser 3 for interactive mini-game boards and game rendering.
- Responsive layered visual assets for mobile/desktop environments.
- Reusable pixel-art graphics, animation assets, avatars and multilingual Spanish/English UI.
- Docker or static front-end deployment is plausible in the project stack; exact production configuration should be confirmed.

System architecture and component boundaries

- Portal shell: screen progression, avatar selection and central tavern/game-selection hub.
- Mini-game registry: route game identifiers to independent state, components and assets.
- Rendering: Phaser for interactive boards while the surrounding interface can be React-driven.
- Localization: common labels, tutorials and validation messages separated from game rules.
- Asset composition: layered backdrops, frames, banners, carpet, lantern/candle animations and mobile-specific offsets.

Data model and domain contracts

- Cows and Bulls rules: secret 4-digit number with distinct digits; count correct positions (bulls) versus correct digits in wrong positions (cows).
- Guess history: attempts, validation results, win condition and optional time/score tracking.
- Hidden Tiles: game-specific tile or card state, moves, completion feedback and sprite rendering.
- Player avatar and language preferences; avoid overclaiming accounts or long-term progress unless verified.

Primary application flows

- Enter the illustrated world and choose an avatar.
- Select a game from the tavern hub, then navigate into its dedicated experience.

- Receive feedback after each move and complete a win state.
- Return to the hub without losing UI control or leaving panels disabled.

Engineering challenges and tradeoffs

- Layering mobile overlays and responsive imagery without obscuring controls.
- Sprite scaling, win-message layout and contrast across backgrounds.
- Proper cleanup of game overlays and restoration of hub clickability.
- Consistent rules and explanations in both supported languages.
- Accessible interaction for touch, keyboard and differing cognitive abilities.

Available evidence and limits

- Previous work describes a Cows and Bulls implementation, assets and hub/phone visual experiments.
- MathQuest Arcade repository and React/TSX files were discussed.
- An integrated production build, complete game catalog and evidence of learning outcomes have not been independently verified.

Recommended technical validation / next steps

- Extract each mini-game behind a small typed interface and test deterministic rules.
- Add accessible step-by-step tutorials, keyboard support and progress states.
- Audit responsive images and overlays across common viewport sizes.
- Instrument game completion, repeat sessions and user comprehension without collecting unnecessary child data.
- Develop educational efficacy testing before marketing claims about learning improvements.

Recruiter interview prompts

- How is each game isolated from the parent shell?
- How do you test Bulls and Cows rules and game-result consistency?
- How are touch, keyboard and overlay focus handled?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.

Career Town (CV RPG) | Technical Case Study

Interactive resume / portfolio | Stage: Concept, world assets, Tiled configuration and character designs; complete public game unverified

Technical summary

Career Town is an interactive RPG-style professional portfolio in which a recruiter explores a small town. The candidate’s education, skills, work and projects appear as discoverable buildings instead of being confined to a static resume page.

Problem and engineering context

Traditional portfolios place many technologies and project details into long scrollable pages. A visually navigable professional map can create stronger differentiation and invite exploration, but must retain straightforward accessibility for busy reviewers.

Technology stack

- Tiled for orthogonal tile map authoring and layered placement.
- Pixel-art spritesheets for recruiter animation and environmental elements.
- Phaser 3 as the intended game runtime for movement and scene interactions.
- Responsive web shell and an ordinary non-game navigation path should accompany the RPG.
- Project materials include 30x24 tile map planning, stone ground, buildings, grass overlay and decoration layers.

System architecture and component boundaries

- Scene entry: accessible intro screen with the option to explore or use a standard project listing.
- Map subsystem: authored Tiled layers, passable paths, building interaction zones and decor.
- Entity subsystem: player sprite animation, collision, navigation and camera.
- Content subsystem: building IDs resolve to portfolio data, technical case studies and downloadable PDFs.
- Fallback navigation: recruiters can access projects, resume and contact information without gameplay.

Data model and domain contracts

- Tile map and layers: StoneGround, Buildings, Grass and decorative elements.
- Building metadata: ID, interaction bounds, title, category and content link.
- Recruiter character: animated movement frames and scale tied to scene proportions.
- Projects registry: descriptions, technology stack, external links and separate technical/business PDFs.

Primary application flows

- Land on the portal with immediately identifiable navigation options.
- Optionally control a character to approach a building.
- Enter a portfolio presentation or open the corresponding project details.

- Return to map, navigate to education/contact or switch to an accessible standard view.

Engineering challenges and tradeoffs

- Character-to-building scale and shadow alignment were previously identified as art consistency issues.
- Large map textures and layered transparency can create rendering and performance costs.
- Keyboard/touch navigation, collision and loading must be reliable.
- The RPG experience cannot hide important career facts from recruiters unwilling to play.
- Metadata and SEO need conventional pages beyond the canvas rendering surface.

Available evidence and limits

- Extensive map, art, tileset and responsive intro design discussions are available.
- There is no verified public production build or recruiter usability testing in the supplied materials.
- A separate Project Showcase portal is being developed as an immediate conventional entry point.

Recommended technical validation / next steps

- Finish one high-quality navigable map section with one interactive building.
- Implement accessible direct links to every project and downloadable PDF.
- Test recruiter task completion (find stack, resume, GitHub, contact) under two minutes.
- Optimize assets and input controls across desktop/mobile.
- Release progressively while keeping the conventional portfolio live.

Recruiter interview prompts

- How do Tiled layer metadata and Phaser collision interact?
- What is the conventional fallback if gameplay fails?
- How do you keep project content synchronized with RPG buildings?

Disclosure: This report intentionally distinguishes observed discussions from proposed engineering improvements. Tests, benchmarks, revenue, user adoption and security compliance are not asserted without supporting evidence.